Text and Images as Data in Social Science

AI and Economics Summer School

Andrea Ciccarone

ETH Zurich

August 31, 2026

Why Do Text and Images Need a Representation?

To study text, images, or video, we first need to represent the raw information numerically:

$$r_i \xrightarrow{\text{representation } f} x_i \in \mathbb{R}^d \xrightarrow{\text{statistical model } g} \widehat{y}_i.$$

- Text had natural units, such as words and phrases, that could be counted in a vocabulary
- Images and video were harder to represent because pixels have no immediate social-science meaning
- Modern embeddings learn numerical representations of each modality and can place them in a common semantic space

We begin with the traditional text-as-data workflow, then introduce learned representations.

Outline

1. Text as Data
2. Image as Data
3. Application: Measuring Partisanship in Images and Text
4. Beyond CLIP: Multimodal Representations

A Standard Text Analysis Workflow

Most text-as-data applications follow three steps:

1. Represent raw documents $d \in \mathcal{D}$ as a numerical array X
2. Map X into a predicted or latent measure $\widehat{Y}$
3. Use $\widehat{Y}$ in descriptive or causal analysis

- The collection of documents $\mathcal{D}$ is the **corpus**
- For example, newspaper articles can be represented numerically, classified by sentiment, and then used to study changes in political tone

The first problem is turning the corpus into a useful X.

Text as Data: Building the Representation

Each document contains tokens drawn from a vocabulary $\mathcal{V}$:

$$d_i \longrightarrow (w_{i1}, \dots, w_{iT_i}), \qquad \mathcal{V} = \{v_1, \dots, v_V\}.$$

- **Tokenize**: choose words, phrases, *n*-grams, or subword pieces as the units of analysis
- **Normalize**: decide how to treat capitalization, punctuation, numbers, and spelling
- **Reduce features**: remove stopwords, stem words, or retain them when the research question requires them
- **Construct** X: represent every document using the same vocabulary

These choices determine which textual distinctions the statistical model can use.

Bag of Words: Counting Words

Choose a vocabulary and count how often each term appears in each document.

Documents

d_1 "tax cuts create jobs"

d_2 "jobs create growth"

d_3 "jobs jobs jobs"

$\Longrightarrow$

Document–term matrix X

	tax	**cuts**	**create**	**jobs**	**growth**
d_1	1	1	1	1	0
d_2	0	0	1	1	1
d_3	0	0	0	**3**	0

$$x_{iv} = \text{count of term } v \text{ in document } i, \qquad X \in \mathbb{R}^{N \times V}.$$

The vector records word frequencies. It contains no information about word order, syntax, or context.

TF–IDF: Reweighting Word Counts

For term v in document i:

$$\mathrm{tfidf}_{iv} = \underbrace{\frac{\mathrm{count}_{iv}}{\sum_{v'} \mathrm{count}_{iv'}}}_{\text{term frequency}} \times \underbrace{\log\left(\frac{N}{\#\{i' : x_{i'v} > 0\}}\right)}_{\text{inverse document frequency}}.$$

- Words with high frequency in one document and low frequency in the corpus receive more weight
- Words appearing everywhere receive little weight
- Cosine similarity between tf–idf vectors is a strong retrieval baseline
- TF–IDF changes the weights. The representation remains nonsemantic

What Can We Do with the Representation?

Once the corpus has become a document-feature matrix X, there are three common approaches:

1. **Dictionary based**: define a known mapping from words to a score
   - Example: assign positive, neutral, or negative values to sentiment words
2. **Supervised**: learn the relationship between text features X and an observed outcome Y
   - Example: predict sentiment or party from hand-coded documents
3. **Unsupervised**: summarize X using latent dimensions $\widehat{\theta}$
   - Example: estimate the topics discussed in each document

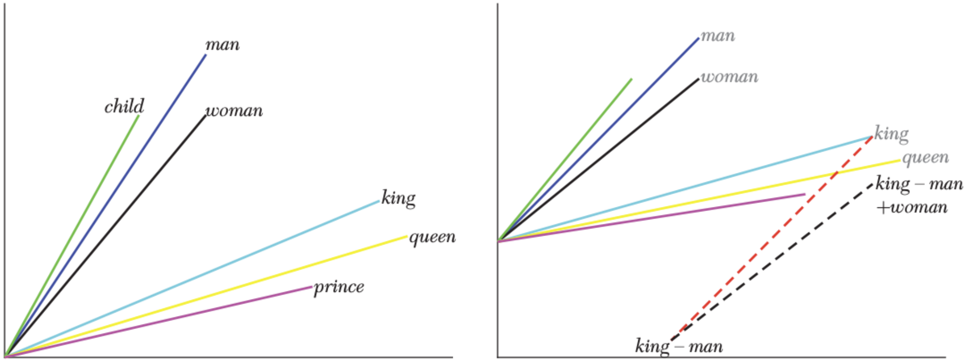
The representation X determines which information each method can recover.

What Do Count Representations Miss?

- **Synonymy**: “job security” and “employment stability” share meaning, not words
- **Polysemy**: “bank” means something different beside “river” and “loan”
- **Negation**: “not guilty” contains the word “guilty”
- **Composition**: “hardly effective” is not the sum of “hardly” and “effective”
- **Long-range context**: meaning can depend on words many tokens away

These failures motivate representations that use context to encode meaning.

Word Embeddings: Meaning as Geometry



$$\text{vec(king)} - \text{vec(man)} + \text{vec(woman)} \approx \text{vec(queen)}.$$

Words that share meaning or relationships occupy related positions in the embedding space.

Mikolov et al. (2013)

How Do We Read an Embedding?

For a raw object $r_i \in \mathcal{X}$, an encoder produces a fixed-length vector:

$$f_\theta : \mathcal{X} \to \mathbb{R}^d, \qquad z_i = f_\theta(r_i) \in \mathbb{R}^d.$$

After normalization, $\tilde{z}_i = z_i / \|z_i\|$, cosine similarity is a dot product:

$$\mathrm{sim}(i, j) = \frac{z_i^\top z_j}{\|z_i\|\|z_j\|} = \tilde{z}_i^\top \tilde{z}_j.$$

- Nearby vectors represent semantically related objects
- Geometric directions can encode concepts and relationships
- A fixed-length z_i can enter prediction, clustering, retrieval, or measurement models
- Dense coordinates require additional interpretation strategies, which we return to in the application

A Fundamental Example of Word Embeddings: BERT

BERT turns each token into a vector that depends on its context.

Input tokens
$(w_1, \dots, w_T)$
token + position vectors

→

BERT encoder
tokens exchange information
through self-attention

→

Contextual vectors
$H = (h_1, \dots, h_T)$
one vector per token

$$h_t = \mathrm{BERT}_\theta(w_1, \dots, w_T)_t.$$

Unlike a static word embedding, h_t depends on **both the token and the surrounding words**.

"the **bank** approved the loan" $\Longrightarrow$ $h_{\text{bank}}^{\text{finance}}$

"the **bank** by the river" $\Longrightarrow$ $h_{\text{bank}}^{\text{geography}}$

Same token, different context $\Rightarrow$ different vector.

Devlin et al. (2019)

How Does BERT Use Context? Self-Attention

Consider one layer and one target token t. Each token is transformed into a **query**, **key**, and **value**:

$$q_t = h_t^{(\ell)} W_Q, \qquad k_j = h_j^{(\ell)} W_K, \qquad v_j = h_j^{(\ell)} W_V.$$

$$\alpha_{tj} = \text{softmax}_j\left(\frac{q_t k_j'}{\sqrt{d_k}}\right), \qquad \widetilde{h}_t = \sum_j \alpha_{tj} v_j.$$

"the **bank** approved the **loan**" $\Longrightarrow$ large $\alpha_{\text{bank,loan}}$

"the **bank** by the **river**" $\Longrightarrow$ large $\alpha_{\text{bank,river}}$

Attention lets "bank" incorporate information from the words that clarify its meaning.

BERT repeats this operation across multiple attention heads and layers: $h_t^{(0)} \rightarrow h_t^{(1)} \rightarrow \cdots \rightarrow h_t^{(L)}$.

How Is BERT Pretrained?

One of BERT's main pretraining objectives is **masked language modeling**.

"the **[MASK]** approved the loan" $\Longrightarrow$ "**bank**"

$$\mathcal{L}_{\mathrm{MLM}}(\theta) = -\sum_{t \in M} \log p_\theta \left(w_t \mid w_{1:T \setminus M} \right).$$

1. Hide a subset of tokens M and use their original values as labels
2. Predict each hidden token from its left and right context
3. Jointly update the embeddings, attention matrices, and other parameters by gradient descent

To predict "bank," the model must use words such as **approved** and **loan**. Repeating this task over large corpora produces contextual representations.

Devlin et al. (2019)

Implementation: BERT in Python

```
from transformers import AutoTokenizer, AutoModel

tokenizer = AutoTokenizer.from_pretrained("bert-base-uncased")
bert = AutoModel.from_pretrained("bert-base-uncased")

batch = tokenizer(texts, padding=True, truncation=True,
                return_tensors="pt")
H = bert(**batch).last_hidden_state # N x T x d

mask = batch["attention_mask"].unsqueeze(-1)
z = (H * mask).sum(1) / mask.sum(1) # N x d
```

$$H_i = (h_{i1}, \ldots, h_{iT_i}) \quad \longrightarrow \quad z_i = \frac{1}{T_i} \sum_{t=1}^{T_i} h_{it}.$$

Mean pooling converts the contextual token vectors in H into one document vector z_i.

A Common Social-Science Application: Topic Models

A topic model summarizes a corpus with two estimated objects:

$$\{d_i\}_{i=1}^{N} \xrightarrow{\text{unsupervised topic model}} \underbrace{\{\beta_k\}_{k=1}^{K}}_{\text{topics}} + \underbrace{\{\theta_i\}_{i=1}^{N}}_{\text{document topic shares}} .$$

Example topics

$$\beta_{\text{jobs}} : \{\text{wages, workers, employment}\},$$

$$\beta_{\text{immigration}} : \{\text{border, asylum, visa}\}.$$

Each β_k describes one recurring theme.

Example document

$$\theta_i = (0.65,\ 0.25,\ 0.10).$$

The entries measure how much document i discusses each topic.

Researchers use $\widehat{\theta}_i$ as measured variables for speeches, news, legislation, and reviews.

How Do Traditional Topic Models Work?

Latent Dirichlet Allocation represents documents as mixtures of K topics:

- Each topic β_k is a probability distribution over the vocabulary
- Each document i has topic shares θ_i
- For each token, choose a topic and then choose a word from that topic

$$z_{ij} \sim \mathrm{Cat}(\theta_i),$$
$$w_{ij} \sim \mathrm{Cat}(\beta_{z_{ij}}).$$

- Output: interpretable topic–word distributions and document–topic shares
- Economists use $\widehat{\theta}_i$ as low-dimensional measured variables

Blei, Ng, and Jordan (2003)

Modern Topic Models Can Use Richer Representations

Modern topic modeling involves two familiar objects:

$$d_i \xrightarrow{\text{representation } h_\phi} x_i \xrightarrow{\text{topic model } p_\theta} \widehat{\pi}_i.$$

Document representation

- Word or *n*-gram counts
- Contextual embeddings
- Sparse, labeled concept counts

Topic-model structure

- Each topic is a distribution over features
- Each document is a mixture of topics
- Neural inference for topic shares

A richer representation can enter the familiar topic-model structure.

Biderman et al. (2026)

Example: Replacing Words with Concepts

"Working night shift can be difficult especially long-term, adapting your social life to a nighttime world can be seriously difficult for some people."

Count	Labeled SAE concept
13	Job performance / employment
11	Sleep and nighttime activities
7	Scheduling and time management
6	Fatigue and exhaustion
5	Difficulty and challenges

Biderman et al. (2026); GemmaScope SAE

$$a_{ij} \xrightarrow{\text{SAE}} r_{ij}, \qquad c_{iv} = \sum_j \mathbf{1}\{r_{ij,v} > q_v\}.$$

- The language model and SAE are pretrained and frozen
- Each column of c_i counts a labeled semantic concept
- The topic model retains the familiar count-based structure with a richer vocabulary

Do the Estimated Topics Make Sense?

Four topics estimated from Glassdoor reviews using SAE concept counts:

Financial performance and strategy

Financial performance; business strategy; employee evaluation

Retail and service work

Food preparation; brands and stores; online shopping and customer service

Healthcare careers and growth

Health-profession requirements; mentorship; growth opportunities

Leave rights and scheduling

FMLA; schedules and time management; job responsibilities

The unsupervised model recovers recognizable dimensions of workplace experience.

Biderman et al. (2026)

Taking Stock: Text as Data

- **Counts and dictionaries** give us transparent, named features
- **Embeddings** capture semantic information in coordinates that require interpretation
- **Dimensionality reduction** turns a high-dimensional representation into a smaller set of useful variables
- **Pretrained representations** can enter familiar tools such as topic models

New representations can be mixed with traditional statistical machinery to match the research question.

Outline

Why Are Images Harder to Measure?

Campaigns, news, protests, organizations, and markets all produce visual evidence.

$$I_i \in [0, 255]^{H \times W \times 3}, \qquad 224 \times 224 \times 3 = 150{,}528 \text{ raw values.}$$

- Pixel coordinates have no natural social-science interpretation
- Translations, crops, and lighting changes alter the raw values even when the content is similar
- Manual coding is slow and expensive for large image corpora

Computer vision learns stable, useful features from pixels.

Image Preprocessing: What Does the Model See?

Models require a common input size and scale:

$$I_i \xrightarrow{\text{resize / crop}} I_i^{224\times224\times3} \xrightarrow{\text{channel normalization}} \widetilde{I}_i.$$

- **Cropping** may discard informative banners, crowds, or background objects
- **Resolution** determines whether small text and facial expressions remain visible
- **Aspect ratio and color** can preserve or distort substantive visual cues
- Apply augmentation only to training data and keep near-duplicates across a single split

Report preprocessing and test reasonable alternatives: these choices change the empirical object.

What Can We Do with Images?

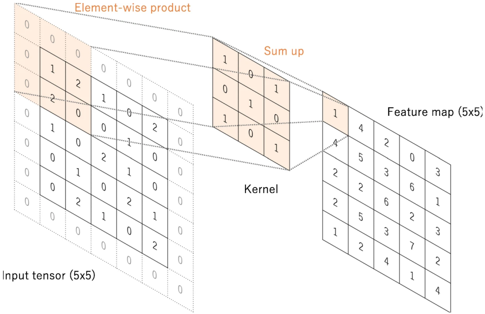
Once an image has a numerical representation, researchers can use it for several tasks:

1. **Classification**: assign one label or score to the entire image
    - Example: predict party, sentiment, land use, or product category
2. **Detection**: locate and classify objects or faces within the image
    - Example: count people, identify symbols, or measure who receives visual attention
3. **Representation**: map the image into features for clustering, retrieval, or downstream models
    - Example: compare visual styles or measure similarity across a large image corpus

These tasks require features that recognize a pattern across different pixel locations.

CNNs Learn Features from Pixels

- A small filter moves across the image
- The same weights are reused everywhere
- Early filters learn edges and textures
- Deeper layers combine them into parts and objects
- Pooling makes features less sensitive to small translations



The model learns its visual vocabulary directly from the training data.

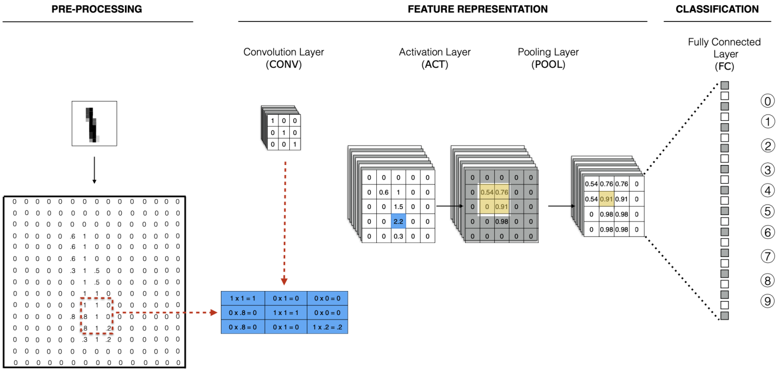
How Does Convolution Work?

For image I and kernel K of height h and width w:

$$F(x, y) = (I * K)(x, y) = \sum_{a=1}^{h} \sum_{b=1}^{w} I(x + a - 1, y + b - 1) K(a, b).$$

- F is a feature map: the response of the same learned pattern at every location
- Weight sharing makes the parameter count independent of image width and height
- Multiple kernels create multiple channels, each responding to a different learned feature

CNNs Learn Features in Layers



- Convolution and pooling blocks produce increasingly abstract features
- Fully connected layers map those features to an outcome
- Remove the final classification layer and the penultimate activations are an **image embedding**

In Practice: Start from a Pretrained Model

1. Take a CNN or vision transformer pretrained on a large image corpus
2. Remove or replace its final classification head
3. Either freeze the encoder and fit a small model on its embeddings, or fine-tune the top layers
4. Validate on images from the actual domain of interest

$$I_i \xrightarrow{f_\theta} z_i \in \mathbb{R}^d \xrightarrow{g_\beta} \widehat{y}_i.$$

Transfer learning is what made image measurement feasible at social-science sample sizes.

Can We Generate Images?

GAN training asks a generator to produce images that a discriminator cannot distinguish from real data:

$$\min_G \max_D \; \mathbb{E}_{x \sim p_{\mathrm{data}}}[\log D(x)] + \mathbb{E}_{z \sim p_z}[\log(1 - D(G(z)))].$$

- The latent vector z becomes a controllable image representation
- Researchers can generate counterfactual stimuli along learned directions
- Diffusion models now dominate image generation. Their latent representations support the same research uses
- Always verify that the generated treatment changes only the intended attribute

(a) Side-by-side mugshot detention morphs with detention probabilities of 0.41 and 0.13 respectively

Goodfellow et al. (2014); Ludwig and Mullainathan (2024)

CLIP: Images and Text in a Shared Space

CLIP trains two encoders:

$$u_i = \frac{f_I(I_i)}{\|f_I(I_i)\|}, \qquad v_i = \frac{f_T(T_i)}{\|f_T(T_i)\|}, \qquad u_i, v_i \in \mathbb{R}^d.$$

- I_i is an image and T_i is its caption
- Both vectors occupy the same learned semantic geometry
- A text prompt can classify an image with no task-specific training
- A researcher can directly compare what an image shows with what a transcript says

Image and text can now enter the same measurement model.

Radford et al. (2021)

How Is CLIP Trained?

In a batch of N matched image–text pairs, define every cross-modal similarity

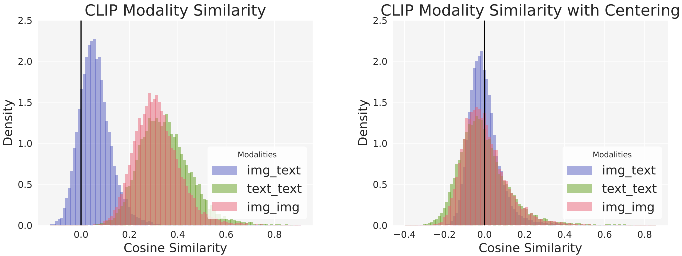
$$s_{ij} = \frac{u_i^\top v_j}{\tau}.$$

CLIP solves the matching problem in both directions:

$$\mathcal{L}_{\mathrm{CLIP}} = -\frac{1}{2N} \sum_{i=1}^{N} \left[\log \frac{e^{s_{ii}}}{\sum_j e^{s_{ij}}} + \log \frac{e^{s_{ii}}}{\sum_j e^{s_{ji}}} \right].$$

- The N diagonal pairs are positives. The $N^2 - N$ off-diagonal pairs serve as negatives
- The loss teaches which image belongs with which text without hand-labeled categories
- τ controls how sharply similarity differences affect the loss

CLIP Still Has a Modality Gap



- Raw image and text embeddings share a geometry and occupy different regions
- SpLICE mean-centers each modality and renormalizes the vectors
- After centering, the distributions align, so one text-defined concept dictionary can describe images and text

Bhalla et al. (2024), Figure 7; MSCOCO images and captions

Taking Stock: Image as Data

- CNNs replaced engineered visual features with learned hierarchical features
- Pretraining and transfer learning made those features usable with modest labeled samples
- GANs and diffusion models turn latent directions into counterfactual images
- CLIP makes visual and textual meaning directly comparable

Image embeddings support comparison, decomposition, and downstream research designs.

Outline

A Running Example: Partisanship in Images and Text

Research task: measure partisan content separately in the images and language of political videos.

- **Labeled corpus:** 15,427 Democratic and Republican television ads from the Wesleyan Media Project, 2016–2020
- **Shared representation:** CLIP maps video frames and transcript segments into the same 512-dimensional space
- **Supervised measure:** train separate image and text classifiers to predict the same sponsor-party label
- **Interpretation:** decompose the dense embeddings into named concepts and test which concepts drive each prediction

The application follows a reusable workflow: **represent** $\rightarrow$ **predict** $\rightarrow$ **interpret**.

Gentzkow and Shapiro (2010); Fowler et al. (2021)

Representing Political Ads in CLIP Space

...

...

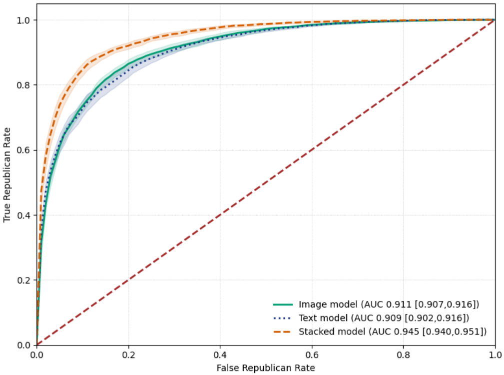
$\downarrow$ $x^1_{img} \in \mathbb{R}^{512}$ $\downarrow$ $x^t_{img} \in \mathbb{R}^{512}$ $\downarrow$ $x^N_{img} \in \mathbb{R}^{512}$

"Dad helped build this business from scratch..." ... *"...protect Medicare and Social Security, especially with COVID..."* ... *"My friends, my family, my community, that's why I'm running for Congress..."*

$\downarrow$ $x^1_{txt} \in \mathbb{R}^{512}$ $\downarrow$ $x^t_{txt} \in \mathbb{R}^{512}$ $\downarrow$ $x^N_{txt} \in \mathbb{R}^{512}$

$$\text{Raw videos} \xrightarrow[\text{pre-trained parameters}]{\text{CLIP}} \{x^1_{mod}, \ldots, x^N_{mod}\} \xrightarrow{\text{aggregation}} \mathbf{v}_{mod} \xrightarrow[\text{trainable parameters}]{\text{neural network}} P(\text{Rep} \mid \mathbf{v}_{mod})$$

How Predictive Are Images and Text?



The same outcome and evaluation metric let us compare how much predictive information each modality carries.

What Else Can We Do with Embeddings?

A dense embedding can support several distinct research operations:

1. **Prediction**: estimate the same construct from text, images, or both
2. **Comparison**: ask which modality contains more signal under the same loss and test set
3. **Transfer**: apply a validated measure to a new corpus with distribution-shift checks
4. **Interpretation**: map dense coordinates onto named concepts and perturb them

Next we use **SpLICE** to turn CLIP vectors into sparse, readable concept weights.

Bhalla et al. (2024)

SpLICE: Representing Images with Named Concepts

We want to describe the content of an image using concepts that we can name.

1. **Define concept vocabularies** relevant to the application:

$$\mathcal{V}_{\text{emotion}} = \{\text{fear, anger, empathy, calm, } \dots\}, \qquad \mathcal{V}_{\text{topic}} = \{\text{immigration, health care, jobs, } \dots\}.$$

2. **Embed every concept with CLIP** and collect the concept vectors in a matrix:

$$C = [c_1, \dots, c_K], \qquad c_k = \sigma(f_T(t_k)).$$

3. **Represent the image embedding** as a sparse, nonnegative linear combination of those concepts:

$$\widehat{x}_i^{\text{img}} = Cw_i = \sum_{k=1}^{K} w_{ik} c_k, \qquad w_{ik} \geq 0.$$

The weights w_{ik} measure how much each named concept contributes to the image representation.

Bhalla et al. (2024)

How Does SpLICE Work?

$$\longrightarrow \quad \mathbf{x}^{\text{img}} = \begin{pmatrix} x_1^{\text{img}} \\ \vdots \\ x_K^{\text{img}} \end{pmatrix}$$

Joint Vocabulary

{**fear, anger..., immigration**}

$$\longrightarrow \quad \mathbf{C} = \begin{bmatrix} c_1^1 & c_1^2 & \cdots \\ \vdots & \ddots & \\ c_K^1 & & c_K^C \end{bmatrix}$$

Sparse Decomposition

$$\longrightarrow \quad \mathbf{w}^{\text{img}} = \begin{cases} \text{immigration} = 0.15 \\ \text{fear} = 0.08 \\ \vdots \\ \text{calm} = 0 \end{cases}$$

Estimating the Concept Weights

For an image or text embedding $\widetilde{x}$, SpLICE estimates nonnegative concept weights:

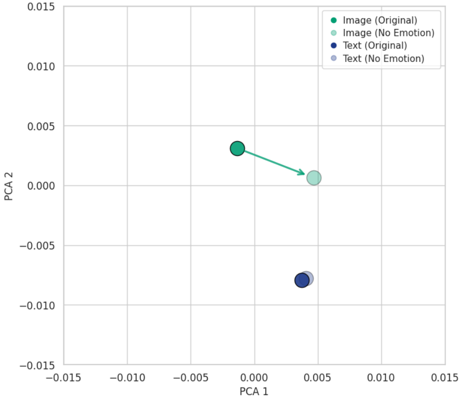
$$w^\star = \arg\min_{w \geq 0} \|Cw - \widetilde{x}\|_2^2 + 2\lambda_1 \|w\|_1, \qquad \widehat{x} = Cw^\star.$$

- $w_c^\star > 0$ means concept c contributes to the sparse reconstruction
- The ℓ_1 penalty limits the number of active concepts, making the representation readable
- The same emotion–topic dictionary is used for images and text
- Sparsity is tuned separately by modality to match reconstruction quality before comparing weights or downstream effects

SpLICE uses CLIP geometry to express an embedding through named semantic components.

Bhalla et al. (2024)

What Happens If We Remove a Concept?



Original reconstruction

$$\widehat{x}_i = C w_i.$$

Remove concept group E

$$\widehat{x}_{i,-E} = C w_{i,-E}.$$

The arrow shows the semantic shift

$$\widehat{x}_i \longrightarrow \widehat{x}_{i,-E}.$$

Pass both through the same predictor

$$\Delta_{i,E} = m(\widehat{x}_i) - m(\widehat{x}_{i,-E}).$$

A larger $|\Delta_{i,E}|$ means the prediction relies more on the removed concepts.

Using Concept Removal to Explain a Model

Once $\widetilde{x}$ is reconstructed as $\widehat{x}_i = Cw_i$, set one concept weight to zero and reconstruct again:

$$w_{i,-c} = w_i \text{ with } w_{ic} = 0, \qquad \widehat{x}_{i,-c} = Cw_{i,-c}.$$

Compare the model's prediction with and without concept c:

$$\Delta_c^{\text{mod}} = \frac{1}{|I_c|} \sum_{i \in I_c} \left[m^{\text{mod}}(\hat{\mathbf{x}}_i) - m^{\text{mod}}(\hat{\mathbf{x}}_{i,-c}) \right]$$

- $\Delta_c > 0$ means concept c raises the model's predicted outcome on average
- Comparing Δ_c across modalities reveals whether image and text models use the same semantic signal
- The quantity describes **model attribution** and has no causal interpretation for viewer responses

What Can Researchers Do with These Methods?

- **Build comparable measures** from text and images using one semantic space
- **Compare information across modalities** under a common outcome and evaluation design
- **Transfer measures across corpora** when the representation generalizes and shift is validated
- **Recover named semantic components** from otherwise opaque embedding coordinates
- **Audit prediction models** through concept removal and cross-modal attribution

These operations apply to many multimodal research questions.

Outline

Beyond CLIP: More Than Two Modalities

CLIP aligns image and text. New multimodal embedding models extend the same idea:

$$z = f_\theta(r), \qquad r \in \{\text{text, image, audio, video, document}\}, \qquad z \in \mathbb{R}^d.$$

- Google's **Gemini Embedding 2** maps all five input types into one unified space
- The same cosine similarity can compare a sentence with a frame, an audio clip, or a video segment
- For long videos, embed overlapping clips and aggregate or retain a time series of semantic vectors
- Researchers can study a multimedia object jointly across its text, image, audio, and video components

Google Gemini API, Gemini Embedding 2 documentation (2026)

Implementation: Gemini Embedding 2 in Python

```
from google import genai
from google.genai import types

client = genai.Client()
video = open("campaign_clip.mp4", "rb").read()

result = client.models.embed_content(
    model="gemini-embedding-2",
    contents=[
        types.Content(parts=[types.Part.from_text(
            text="a political campaign rally")]),
        types.Content(parts=[types.Part.from_bytes(
            data=video, mime_type="video/mp4")]),
    ],
)

z_text, z_video = [e.values for e in result.embeddings]
```

Separate `Content` objects return comparable vectors for each modality.

Google AI for Developers, Embeddings guide (2026)

Working with Multimodal Data

1. Define the construct and the relevant unit: sentence, frame, clip, post, or full document
2. Embed each modality in a shared space and choose an aggregation rule
3. Build the supervised, unsupervised, or similarity-based measure
4. Validate separately by modality and jointly across modalities
5. Preserve timestamps and metadata so the measure can enter an economic design

A unified representation still requires careful choices about measurement and validation.

Recap

1. Text was historically easier because words supplied a natural, interpretable vocabulary
2. Embeddings replaced modality-specific surface features with learned semantic geometry
3. BERT uses context to represent how a token's meaning changes across sentences
4. Sparse labeled concepts support more interpretable text measures and topic models
5. CNNs learn visual features, and CLIP aligns visual and textual meaning
6. Multimodal models extend this alignment to text, images, audio, video, and documents
7. Representation remains a research choice that requires validation before economic analysis